Setting Up a Workflow in Apache Airflow

In this exercise, you will **build and run a simple Apache Airflow DAG** that simulates the orchestration of a basic data pipeline. Your task is to **create a workflow with three tasks**: fetching data, processing it, and storing the result.

You will use Airflow's DAG (Directed Acyclic Graph) structure to **configure dependencies and schedule the execution** of your tasks. This activity will help you **understand how data workflows are designed and automated** using Apache Airflow.

Learning Objectives

- Understand the structure of an Airflow DAG.
- Create a simple Airflow DAG with Python.
- Set task dependencies to define the workflow order.
- Trigger and monitor the workflow in the Airflow UI.

Instructions

Step 1: Set up your Airflow environment

Follow these steps to get started with Airflow locally:

- Open your terminal and run:

  ```
  docker run -d -p 8080:8080 --name airflow -e LOAD_EX=n apache/airflow:2.12.0 standalone
  ```

- Wait for the web server to start, then visit `http://localhost:8080`
- Login with `admin / admin`

Hint: You may need to install Docker before starting.

Step 2: Create your first DAG

Create a new Python file named `simple_dag.py` in your Airflow `dags` folder:

```
from airflow import DAG from airflow.operators.python
import PythonOperator from datetime
import datetime
def fetch_data():
  print("Fetching data...")
def process_data():
  print("Processing data...")
def store_data():
  print("Storing data...")
with DAG(
  dag_id="simple_etl_dag",
  start_date=datetime(2023, 1, 1),
  schedule_interval=None,
  catchup=False,
  description="A simple ETL pipeline in Airflow",
  ) as dag:
      fetch = PythonOperator(
      task_id="fetch_data",
      python_callable=fetch_data
      )
      process = PythonOperator(
      task_id="process_data",
      python_callable=process_data
      )
      store = PythonOperator(
      task_id="store_data",
      python_callable=store_data
      )
      fetch >> process >> store
```

Save and check that your DAG appears in the Airflow UI.

Step 3: Trigger your DAG and monitor the run

- In the Airflow UI, enable your DAG by toggling it on.
- Click “Trigger DAG” to run it manually.

- Use the Graph View or Tree View to follow task execution.

Check that all tasks complete successfully and logs print your messages.

Step 4: Reflect and improve

Answer the following short questions:

- What is the purpose of each task in your DAG?
- How does Airflow ensure the tasks run in order?
- How would you modify this workflow to include data validation?

Deliverables

> - **Python file:** A working `simple_dag.py` with fetch, process, and store tasks.
> - **Screenshot:** DAG graph view showing successful run of all three tasks.
> - **Written reflection:** Answers to the reflection questions in Step 4.

Key Takeaways

- **DAG Structure:** An Airflow DAG defines the order and dependencies of tasks using Python, making workflows transparent and version-controlled.
- **Task Dependencies:** Using the >> operator sets execution order, ensuring tasks like fetch, process, and store run sequentially and correctly.
- **Monitoring and Execution:** The Airflow UI provides real-time visibility into DAG runs, task status, and logs, making it easy to debug and validate workflows.